

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ ЖИВОТНОГО МИРА
НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное
учреждение Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 13.

**Тема: «Проектирование и реализация сервисного слоя с
использованием паттернов Repository, Service Layer,
DTO и внедрения зависимостей»**

Выполнила: Потанина Н.А.,

Студентка группы: 24-ИС.

2026г.

Цель работы:

Сформировать у обучающихся навыки **проектирования и реализации** сервисного слоя с использованием архитектурных паттернов, обеспечивающих слабую связанность, тестируемость и расширяемость.

Вариант №2 «Система промокодов»

Структура проекта:

У:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 13/

- main.py # ГЛАВНЫЙ ФАЙЛ
- Domain Models.py # Модели данных
- Кастомные исключения.py # Исключения
- DTO для промокодов.py # DTO для промокодов
- BookingCreateDTO.py # DTO для бронирований
- Репозиторий для промокодов.py # Репозиторий промокодов
- Репозиторий для использования промокодов.py
- Стратегии скидок.py # Стратегии скидок
- run.bat # Файл для быстрого запуска
- README.md # Описание проекта

```
# main.py

from datetime import datetime, date, timedelta
from dataclasses import dataclass, field
from typing import Optional, List, Dict, Any
from enum import Enum
from abc import ABC, abstractmethod

# ===== ENUMS =====

class BookingStatus(Enum):
    PENDING = "pending"
    CONFIRMED = "confirmed"
    CHECKED_IN = "checked_in"
    CHECKED_OUT = "checked_out"
    CANCELLED = "cancelled"

class PromoCodeType(Enum):
    PERCENTAGE = "percentage"
    FIXED = "fixed"
    FREE_NIGHT = "free_night"

class ActionType(Enum):
    CREATE = "create"
    UPDATE = "update"
    DELETE = "delete"
    CANCEL = "cancel"
    CONFIRM = "confirm"
    APPLY_PROMO = "apply_promo"
    USE_PROMO = "use_promo"

# ===== DOMAIN MODELS =====

@dataclass
class Hotel:
    id: Optional[int]
    name: str
```

1.main.py

```

# Domain Models.py

from dataclasses import dataclass, field
from datetime import date, datetime
from typing import Optional
from enum import Enum

# ===== ENUMS =====

class BookingStatus(Enum):
    PENDING = "pending"
    CONFIRMED = "confirmed"
    CHECKED_IN = "checked_in"
    CHECKED_OUT = "checked_out"
    CANCELLED = "cancelled"

class PromoCodeType(Enum):
    PERCENTAGE = "percentage"      # Процентная скидка
    FIXED = "fixed"                # Фиксированная сумма
    FREE_NIGHT = "free_night"     # Бесплатная ночь

class PromoCodeUsageStatus(Enum):
    ACTIVE = "active"
    USED = "used"
    EXPIRED = "expired"

class RoomType(Enum):
    STANDARD = "standard"
    DELUXE = "deluxe"
    SUITE = "suite"
    PRESIDENTIAL = "presidential"

# ===== DOMAIN MODELS =====

@dataclass
class Hotel:
    """Модель отеля"""

```

2. Domain Models.py

```

# Кастомные исключения.py

# ===== БАЗОВОЕ ИСКЛЮЧЕНИЕ =====

class DomainError(Exception):
    """Базовое исключение домена"""
    def __init__(self, message: str, details: dict = None):
        self.message = message
        self.details = details or {}
        super().__init__(message)

    def __str__(self):
        if self.details:
            return f"{self.message} (Детали: {self.details})"
        return self.message

# ===== ИСКЛЮЧЕНИЯ ДЛЯ ОТЕЛЕЙ =====

class HotelNotFoundError(DomainError):
    """Отель не найден"""
    def __init__(self, hotel_id: int):
        super().__init__(
            f"Отель с ID {hotel_id} не найден",
            details={"hotel_id": hotel_id}
        )

class HotelNameError(DomainError):
    """Некорректное название отеля"""
    def __init__(self, name: str):
        super().__init__(
            f"Некорректное название отеля: '{name}'",
            details={"name": name}
        )

class HotelRatingError(DomainError):
    """Некорректный рейтинг отеля"""
    def __init__(self, rating: float):

```

3. Кастомные исключения.py

```

# DTO для промокодов.py

from pydantic import BaseModel, field_validator, ConfigDict
from datetime import date, datetime
from typing import Optional, Literal

# ===== DTO ДЛЯ СОЗДАНИЯ ПРОМОКОДА =====

class PromoCodeCreateDTO(BaseModel):
    """DTO для создания нового промокода"""
    code: str
    description: str
    type: Literal["percentage", "fixed", "free_night"]
    value: float
    min_booking_amount: float = 0.0
    valid_from: date
    valid_until: date
    max_uses: int = 1

    # Правильная конфигурация для Pydantic V2
    model_config = ConfigDict(
        str_strip_whitespace=True, # Удаляем пробелы в строках
        validate_default=True, # Валидировать значения по умолчанию
        populate_by_name=True, # Позволяет использовать имена полей
        from_attributes=True # Поддержка ORM объектов
    )

    @field_validator('value')
    @classmethod
    def validate_value(cls, v, info):
        """Валидация значения скидки в зависимости от типа"""
        data = info.data
        promo_type = data.get('type')

        if promo_type == 'percentage':
            if v <= 0 or v > 100:
                raise ValueError('Процентная скидка должна быть между 1 и 100')

```

4. DTO для промокодов.py

```

# BookingCreatedDTO.py

from pydantic import BaseModel, field_validator, ConfigDict
from datetime import date, datetime
from typing import Optional

# ===== DTO ДЛЯ СОЗДАНИЯ БРОНИРОВАНИЯ =====

class BookingCreateDTO(BaseModel):
    """DTO для создания нового бронирования"""
    room_id: int
    guest_name: str
    guest_email: str
    check_in: date
    check_out: date

    # Используем ConfigDict вместо class Config
    model_config = ConfigDict(
        str_strip_whitespace=True, # Удаляем пробелы в строках
        validate_default=True, # Валидировать значения по умолчанию
        populate_by_name=True, # Позволяет использовать имена полей
        from_attributes=True # Поддержка ORM объектов
    )

    @field_validator('check_out')
    @classmethod
    def validate_dates(cls, v, info):
        """Валидация дат бронирования"""
        data = info.data
        if 'check_in' in data:
            check_in = data['check_in']
            if v <= check_in:
                raise ValueError('Дата выезда должна быть позже даты заезда')
            if (v - check_in).days > 30:
                raise ValueError('Бронирование не может превышать 30 дней')
        return v

```

5. BookingCreatedDTO.py

```

# Репозиторий для промокодов.py

from typing import List, Optional, Dict
from datetime import date, datetime
from enum import Enum
from dataclasses import dataclass, field

# ===== ВСПОМОГАТЕЛЬНЫЕ КЛАССЫ (дублируем для автономной работы) =====

class PromoCodeType(Enum):
    PERCENTAGE = "percentage"
    FIXED = "fixed"
    FREE_NIGHT = "free_night"

@dataclass
class PromoCode:
    """Модель промокода"""
    id: Optional[int]
    code: str
    description: str
    type: PromoCodeType
    value: float
    valid_from: date
    valid_until: date
    min_booking_amount: float = 0.0
    max_uses: int = 1
    used_count: int = 0
    is_active: bool = True
    created_at: datetime = field(default_factory=datetime.now)

    def __post_init__(self):
        if self.code:
            self.code = self.code.strip().upper()

    def is_valid(self, current_date: Optional[date] = None) -> bool:
        if current_date is None:
            current_date = date.today()

```

6. Репозиторий для промокодов.py

```

# Репозиторий для использования промокодов.py

from typing import List, Optional, Dict
from datetime import datetime
from enum import Enum
from dataclasses import dataclass, field

# ===== ВСПОМОГАТЕЛЬНЫЕ КЛАССЫ (дублируем для автономной работы) =====

class PromoCodeUsageStatus(Enum):
    ACTIVE = "active"
    USED = "used"
    EXPIRED = "expired"

@dataclass
class PromoCodeUsage:
    """Модель использования промокода"""
    id: Optional[int]
    promo_code_id: int
    booking_id: int
    user_email: str
    discount_amount: float
    used_at: datetime = field(default_factory=datetime.now)
    status: PromoCodeUsageStatus = PromoCodeUsageStatus.USED

    def __post_init__(self):
        if self.discount_amount < 0:
            raise ValueError("Сумма скидки не может быть отрицательной")
        if not self.user_email or "@" not in self.user_email:
            raise ValueError("Некорректный email")

class BaseRepository:
    """Базовый абстрактный репозиторий"""
    def get_by_id(self, id: int):
        raise NotImplementedError

    def get_all(self, **filters):

```

7. Репозиторий для использования промокодов.py

```

# Стратегии скидок.py

from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime, date
from typing import Optional, List
from enum import Enum

# ===== ВСПОМОГАТЕЛЬНЫЕ КЛАССЫ =====

class PromoCodeType(Enum):
    PERCENTAGE = "percentage"
    FIXED = "fixed"
    FREE_NIGHT = "free_night"

class BookingStatus(Enum):
    PENDING = "pending"
    CONFIRMED = "confirmed"
    CHECKED_IN = "checked_in"
    CHECKED_OUT = "checked_out"
    CANCELLED = "cancelled"

@dataclass
class PromoCode:
    """Модель промокода"""
    id: Optional[int]
    code: str
    description: str
    type: PromoCodeType
    value: float
    valid_from: date
    valid_until: date
    min_booking_amount: float = 0.0
    max_uses: int = 1
    used_count: int = 0
    is_active: bool = True
    created_at: datetime = field(default_factory=datetime.now)

```

8. Стратегии скидок.py

```

# main.py |

from dataclasses import dataclass, field
from datetime import datetime, date, timedelta
from typing import Optional, List, Dict, Any
from enum import Enum
from abc import ABC, abstractmethod

# ===== ENUMS =====

class BookingStatus(Enum):
    PENDING = "pending"
    CONFIRMED = "confirmed"
    CHECKED_IN = "checked_in"
    CHECKED_OUT = "checked_out"
    CANCELLED = "cancelled"

class PromoCodeType(Enum):
    PERCENTAGE = "percentage"
    FIXED = "fixed"
    FREE_NIGHT = "free_night"

class ActionType(Enum):
    CREATE = "create"
    UPDATE = "update"
    DELETE = "delete"
    CANCEL = "cancel"
    CONFIRM = "confirm"
    APPLY_PROMO = "apply_promo"
    USE_PROMO = "use_promo"

# ===== DOMAIN MODELS =====

@dataclass
class Hotel:
    id: Optional[int]
    name: str

```

9. python main.py

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

:\Users\Student237>cd "Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 13"

:\Users\Student237>python main.py

:\Users\Student237>python "Y:/24 ИСиП 2025/УП02/Группа 2/Потанина Н.А/День 13/main.py"
=====
СИСТЕМА УПРАВЛЕНИЯ БРОНИРОВАНИЯМИ С ПРОМОКОДАМИ И АУДИТОМ
=====

АГ 1: СОЗДАНИЕ ОТЕЛЯ И НОМЕРОВ
=====
Создан отель: Grand Hotel (рейтинг: 4.5)
Номер 101: вместимость 2, цена 2500.0 у.е./ночь
Номер 102: вместимость 4, цена 3500.0 у.е./ночь
Номер 103: вместимость 2, цена 2800.0 у.е./ночь
=====

АГ 2: СОЗДАНИЕ ПРОМОКОДОВ
=====
Процентный: SUMMER2026 - 15% (мин. сумма 5000 у.е.)
Фиксированный: FIXED1000 - 1000 у.е. (мин. сумма 3000 у.е.)
Бесплатная ночь: FREE1NIGHT - 1 ночь (мин. сумма 2000 у.е.)
=====

АГ 3: СОЗДАНИЕ БРОНИРОВАНИЙ
=====
Бронирование #1: Иван Петров
Номер 101, даты: 2026-07-15 - 2026-07-20
Стоимость: 12500.0 у.е.
Бронирование #2: Мария Иванова
Номер 102, даты: 2026-08-10 - 2026-08-15
Стоимость: 17500.0 у.е.
=====

АГ 4: ПРИМЕНЕНИЕ ПРОМОКОДОВ
=====
-- Применение SUMMER2026 к бронированию #1 ---
Скидка: 1875.0 у.е.
Итоговая стоимость: 10625.0 у.е.
Экономия: 15.0%

-- Применение FIXED1000 к бронированию #2 ---
Скидка: 1000 у.е.
Итоговая стоимость: 16500.0 у.е.
=====

АГ 5: ПРОВЕРКА АУДИТ-ЛОГА
=====
История изменений для бронирования #1:
-----
2026-06-19 09:40:36 | Применение промокода | admin@example.com
  → total_price: 12500.0 → 10625.0
  → {'promo_code': 'SUMMER2026', 'discount': 1875.0}
2026-06-19 09:40:36 | Создание | admin@example.com
=====

АГ 6: ДОПОЛНИТЕЛЬНЫЕ СЦЕНАРИИ
```

```

=====
ДИАГ 5: ПРОВЕРКА АУДИТ-ЛОГА
=====

История изменений для бронирования #1:
-----
2026-06-19 09:40:36 | Применение промокода | admin@example.com
  → total_price: 12500.0 → 10625.0
  → {'promo_code': 'SUMMER2026', 'discount': 1875.0}
2026-06-19 09:40:36 | Создание | admin@example.com
-----

ДИАГ 6: ДОПОЛНИТЕЛЬНЫЕ СЦЕНАРИИ
=====

--- Попытка применения несуществующего промокода ---
Ошибка: Промокод 'INVALID_CODE' не найден

--- Попытка применения промокода к несуществующему бронированию ---
Ошибка: Бронирование 999 не найдено

--- Статистика использования промокодов ---
SUMMER2026: использован 1 раз, общая скидка 1875.0 у.е.
FIXED1000: использован 1 раз, общая скидка 1000 у.е.
FREE1NIGHT: использован 0 раз, общая скидка 0 у.е.

--- Активные промокоды ---
SUMMER2026: Летняя скидка 15% (осталось 99 использований)
FIXED1000: Фиксированная скидка 1000 у.е. (осталось 49 использований)
FREE1NIGHT: Одна бесплатная ночь (осталось 30 использований)

--- Все бронирования ---
#1: Иван Петров - 10625.0 у.е., промокод: 1
#2: Мария Иванова - 16500.0 у.е., промокод: 2
-----
ДЕМОНСТРАЦИЯ ЗАВЕРШЕНА
=====

C:\Users\Student237>_

```

10. Запуск программы

Ответы на контрольные вопросы:

1. Какая основная цель выделения сервисного слоя?

Выделить бизнес-логику в отдельный слой, чтобы она не смешивалась с:

HTTP-запросами (контроллеры)

Работой с базой данных (репозитории)

Зачем:

Код становится понятнее и проще поддерживать

Бизнес-логику можно переиспользовать (из API, CLI, очередей)

Легче тестировать (без зависимостей от HTTP и БД)

2. Чем отличается DTO от Domain Model?

DTO	Domain Model	
Назначение	Передача данных между слоями	Бизнес-сущность с логикой
Где используется	На входе/выходе API	Внутри сервисного слоя
Содержит логику?	Нет (только валидация)	Да (методы, правила)
Пример	BookingCreateDTO	Booking (с методом can_cancel())

3. Что такое Unit of Work и зачем он нужен?

Unit of Work — паттерн, который управляет транзакциями.

Зачем: Все изменения в рамках одной операции либо сохраняются, либо откатываются.

Обеспечивает атомарность операций.

4. В чем разница между паттернами Repository и Service?

Repository	Service	
Что делает	Работает с данными (CRUD)	Содержит бизнес-логику
С кем работает	С БД / хранилищем	С репозиториями и другими сервисами
Пример	booking_repo.get_by_id()	booking_service.cancel_booking()

5. Какую проблему решает внедрение зависимостей (DI)?

Проблема: Класс жестко привязан к своим зависимостям.

Без DI:

```
python
class BookingService:
    def __init__(self):
        self.repo = BookingRepository()
```

С DI:

python

```
class BookingService:
```

```
    def __init__(self, repo: BookingRepository):  
        self.repo = repo
```

Что дает:

Легко заменять зависимости (например, на моки в тестах)

Код становится гибче и переиспользуемее

6. Почему нельзя использовать бизнес-логику в контроллерах?

Проблемы:

1. Дублирование кода — логика повторяется в разных контроллерах
2. Сложность тестирования — нужно поднимать HTTP-сервер
3. Трудность переиспользования — нельзя вызвать логику из CLI или очереди
4. Нарушение SRP — контроллер занимается и HTTP, и логикой

Правильно: контроллер → вызывает сервис → сервис содержит логику

7. Какие типы исключений должны быть в сервисном слое?

Должны быть свои кастомные исключения для бизнес-ошибок:

Базовое исключение

DomainError

Конкретные ошибки

RoomNotFoundError

RoomNotAvailableError

BookingConflictError

InvalidDatesError

PromoCodeNotFoundError

PromoCodeExpiredError

PromoCodeLimitExceededError

BookingNotFoundError

8. Как протестировать сервис без подключения к реальной БД?

Использовать моки (Mock-объекты):

```
from unittest.mock import Mock
```

```
def test_booking_service():
```

```
    # 1. Создаем мок репозитория
```

```
    mock_repo = Mock()
```

```
    mock_repo.get_by_id.return_value = Booking(...)
```

```
# 2. Передаем мок в сервис (DI)
service = BookingService(mock_repo)
```

```
# 3. Тестируем логику
result = service.cancel_booking(1)
```

```
# 4. Проверяем вызовы
mock_repo.update.assert_called_once()
```

Преимущества:

Не нужна реальная БД

Тесты работают быстро

Можно проверить все сценарии (включая ошибки)

Вывод: Задание выполнено полностью в соответствии с требованиями варианта 2. Реализована система промокодов с тремя типами скидок, всеми необходимыми проверками, корректным расчетом скидок и учетом использований. Архитектура соответствует принципам слабой связанности и высокой тестируемости. Система готова к интеграции с API, CLI или очередями сообщений.